

Software Dependability

Performance Testing

Intro

When developing software systems, some non-functional requirements cannot be ignored:

- **Speed.** Does the application respond quickly enough for the intended users?
- **Scalability.** Will the application handle the expected user load and beyond?
- **Stability.** Is the application stable under expected and unexpected user loads?
- **Confidence.** Are you sure users will have a positive experience on the go-live day?

Speed

Users

- What is the experience of the users with the system?
- What is the users' perception of speediness (psychology)?
- What are the different users' usage habits?

System

- What hardware is required to run and monitor the system?
- What network bandwidth and latency do users need to connect to the system?
- What non-functional requirements are requested?



The deployment costs are based on two factors



- the knowledge that designers have of users;
- the system constraints.

Scalability

Users

- How many users should be active simultaneously before the system gets “slow”?
- How many users should be active simultaneously before the system stops working?
- How many users should I expect today?
- How many users should I expect before the next update?

System

- What is the capability of the database?
- What is the capability of the file server?
- What is the capability of the application server?
- What is the capability of the backup server?
- What are the expected data growth rates?

Stability

Users

- What happens if there are more users than expected?
- What happens if all users do the same thing at the same time?
- What happens if a user gets disconnected?

System

- What is the behavior of the system in case of a Denial of Service attack?
- What happens if the server goes down?
- What happens if the system records too many orders for the same thing?

Confidence

Ability to measure performance-related aspects of your system

If you can measure what the performance of the system will be,

- you can assess risks;
- you can make informed decisions;
- you can plan for the future in terms of infrastructure and new staff;
- you can sleep the night before the go-live day!

The peace of mind that the system will work on go-live days justifies the cost of performance testing.

What is Performance Testing About?

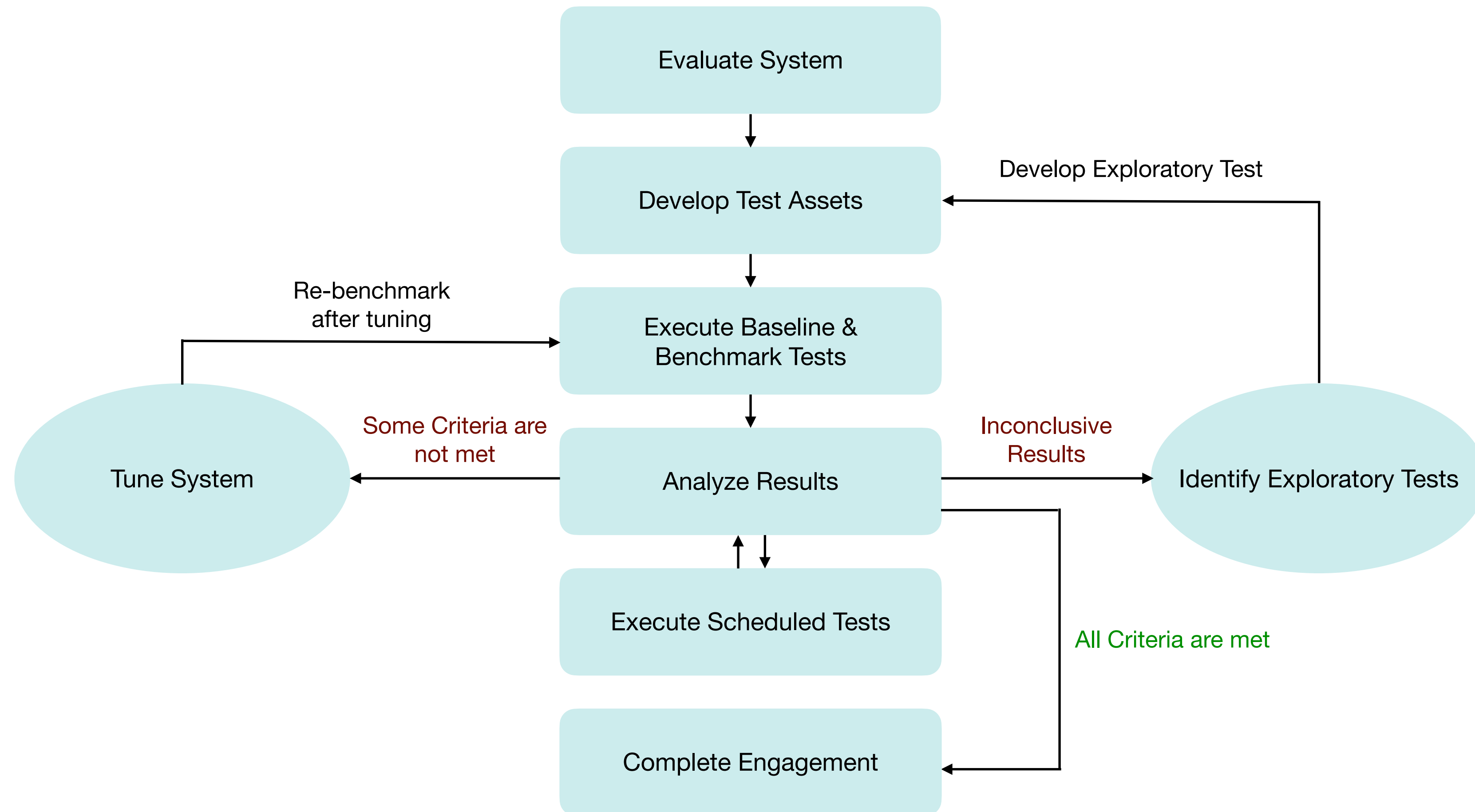
Performance engineering. The process by which software is tested and tuned to **realize the required performance**. It aims at optimizing the most critical application performance trait: user experience.

Performance testing. The process by which software is tested to determine **the current system performance**. It aims to gather information about current performance, create benchmarks, and evaluate degradations.

Performance validation. The process by which software is tested to determine if it meets **pre-existing performance requirements**. It aims at assessing compliance.

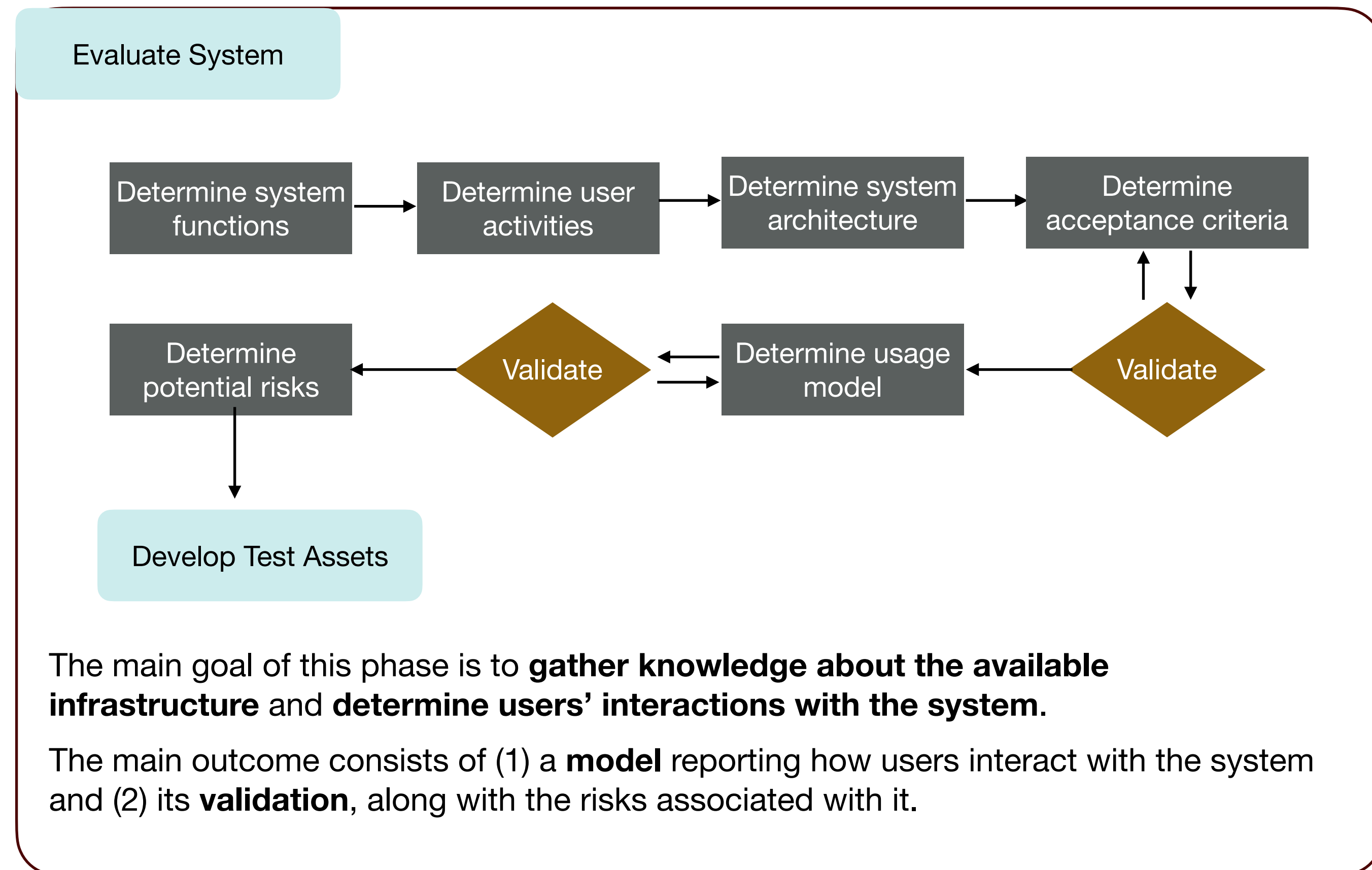
Performance Engineering

The performance engineering process follows the steps below.



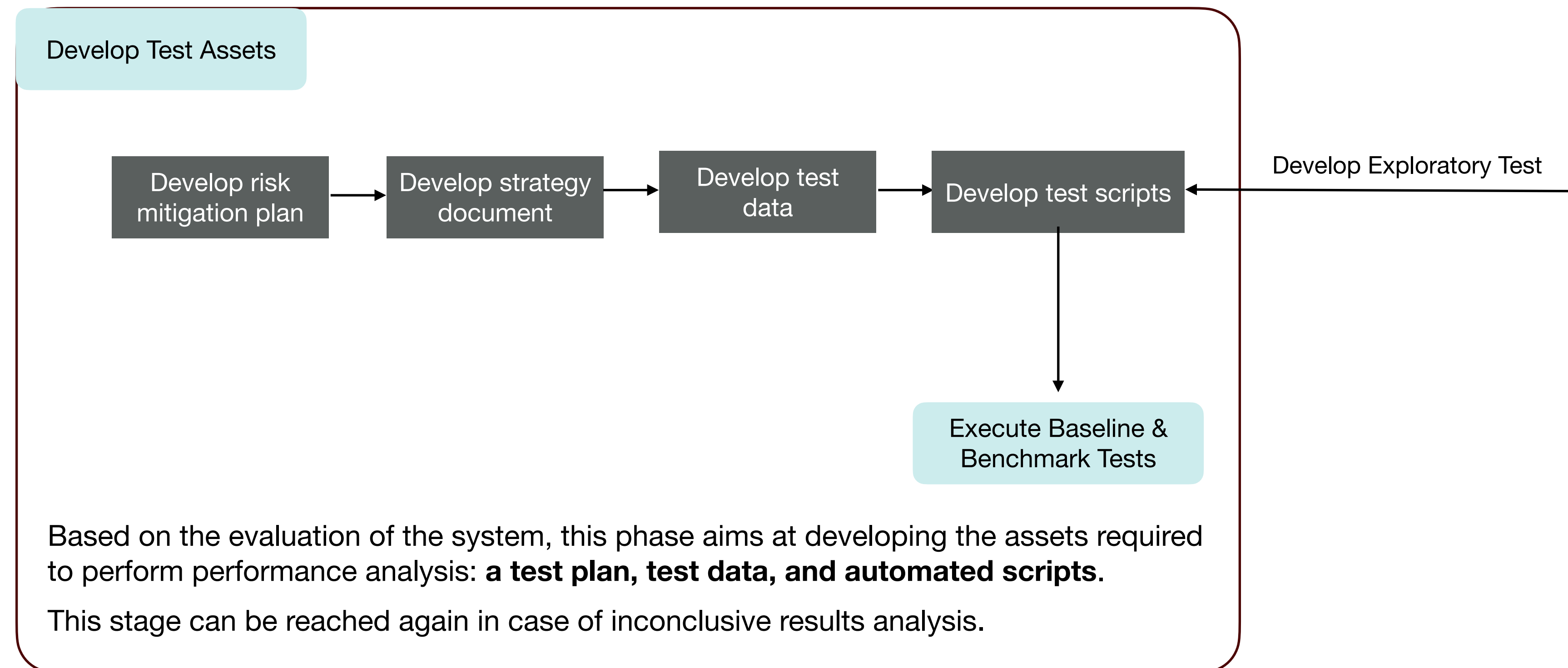
Performance Engineering

The performance engineering process follows the steps below.



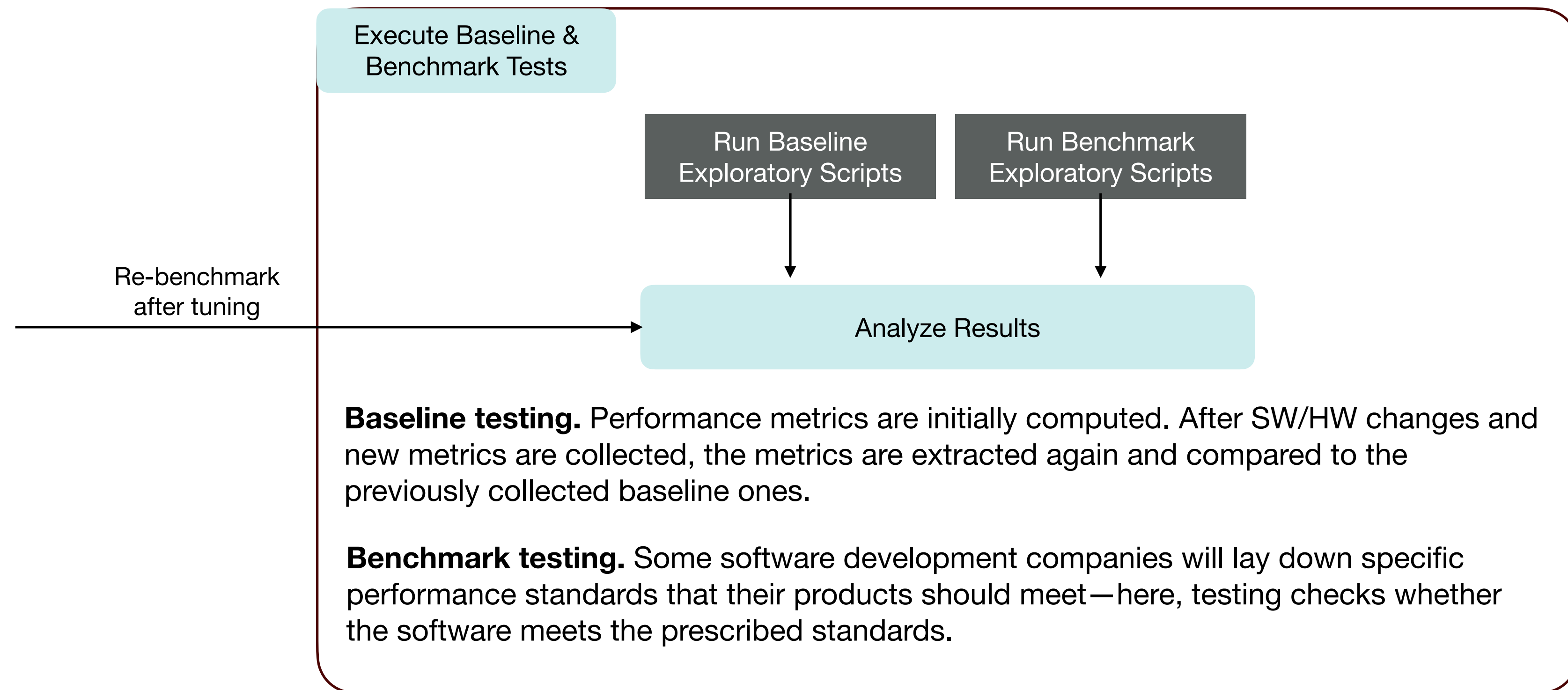
Performance Engineering

The performance engineering process follows the steps below.



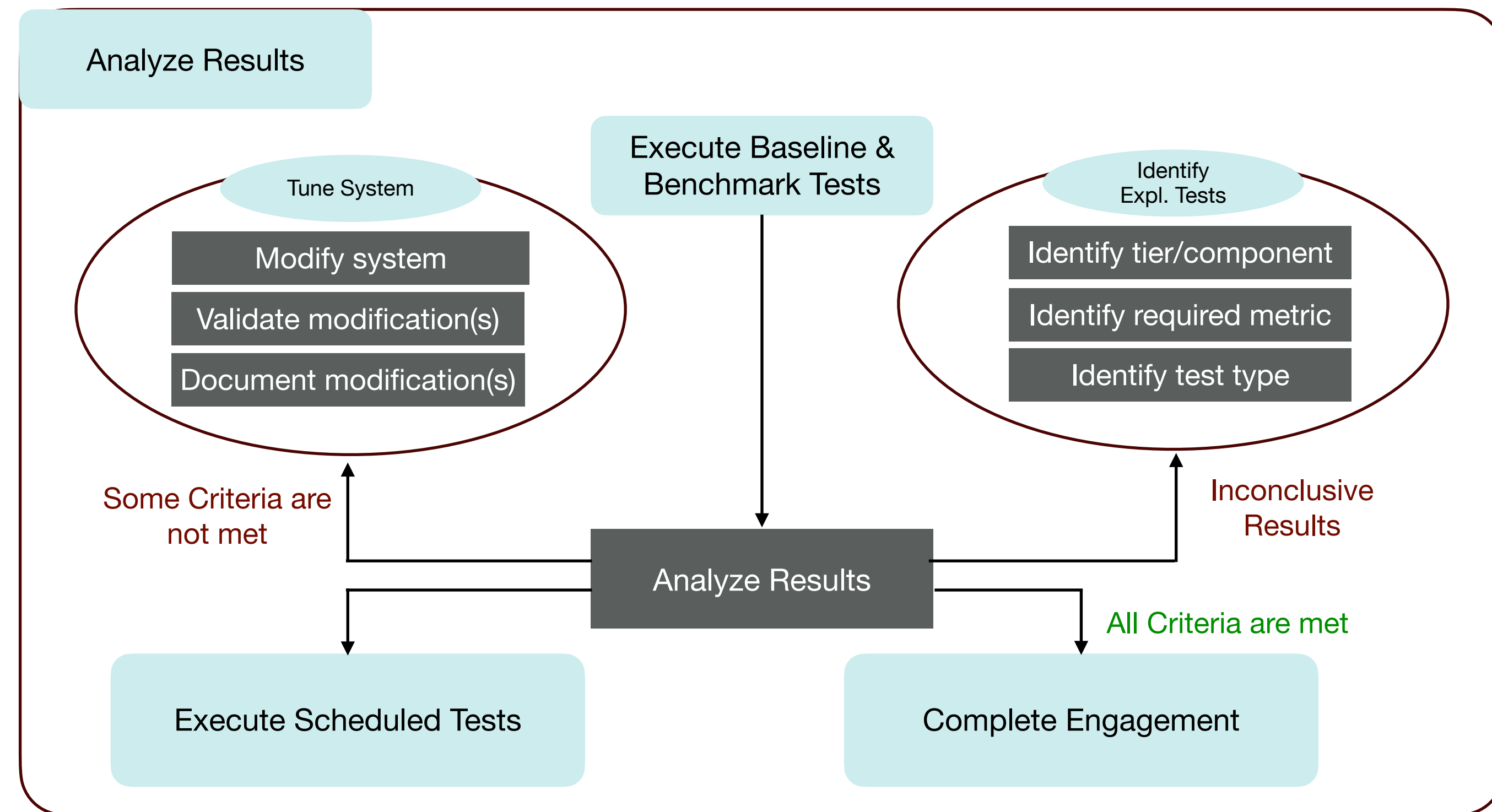
Performance Engineering

The performance engineering process follows the steps below.



Performance Engineering

The performance engineering process follows the steps below.



Performance Engineering

The performance engineering process follows the steps below.

Complete Engagement

Once all criteria are met, some actions are performed to complete the process

Documentation

The process must be documented:

- Actual performance test results;
- Tuning summary;
- Known not tuned bottlenecks;
- Other supporting information;
- Recommendations for future iterations.

Package Test Assets

All results must be archived:

- Test scripts and data;
- Accompanying test documents;
- Store packages for future iterations.

Performance Testing Factors

Performance testing determines the **speed** (i.e., **effectiveness**) of a **computer, network, software program, or device**. Before going into the details, it is worth discussing the factors that govern performance testing.

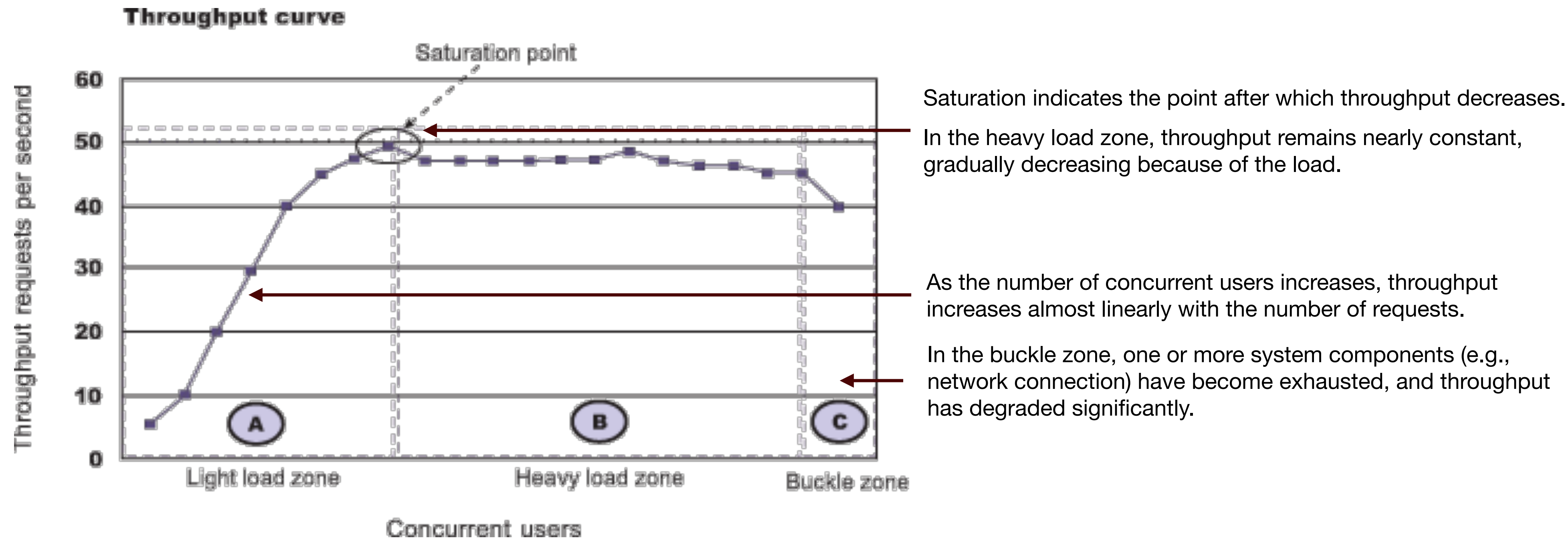
Throughput indicates the **capability of a product to handle multiple transactions in a given period**.

It is the **number of requests/business transactions** in a specified period, and **it depends on several factors like:**

- **Specifications** of the host machine;
- **Degree of parallelism** in both hardware and software support;
- **Processing overhead** of software;
- **Types of transactions** being processed.

Performance Testing Factors

Performance testing determines the **speed** (i.e., **effectiveness**) of a **computer, network, software program, or device**. Before going into the details, it is worth discussing the factors that govern performance testing.



Performance Testing Factors

Performance testing determines the **speed** (i.e., **effectiveness**) of a **computer, network, software program, or device**. Before going into the details, it is worth discussing the factors that govern performance testing.

The **response time** is defined as the **delay between the request and the first response of the product**.

Typically, response time increases proportionally to the user load - i.e., the higher the number of users, the lower the system responsiveness, the higher the response time.

Consumption quantifies the **memory** (e.g., heap usage, GC times), **CPU** (e.g., instructions per cycle), **locks** (e.g., holding times), and **I/O requests** (e.g., to file system, to network) consumed by users in a period.

Performance Testing

Performance Testing is the set of testing activities to evaluate the response time, throughput, and usage of the system to perform its functions in comparison to different versions of the same product or competitive products.

It is conducted **to derive benchmark statistics of the system.**

Performance testing applied to the whole system

- **Load testing** exercises the system under test by feeding it the largest tasks it can operate with;
- **Stress testing** exercises the system excessively to determine the failure point;
- **Spike testing** exercises the system with a sudden rocket of load;
- **Soak testing** exercises the system by continuously testing it with constant load over a long period.

Performance testing applied to individual components

- **Microbenchmarking** is equivalent to unit tests but for performance. It aims to measure the performance of small software components (e.g., methods).

Performance testing applied to the whole system

1. Bring up a production-like system;
2. Create a test plan;
3. Generate load (# of concurrent users, # of transactions) over time;
4. Measure and collect performance metrics;
5. Generate test results (e.g., graphs, CSV);
6. Analyze test results, typically through statistics.

Plenty of tools available make it possible to:

- Perform load testing of multiple applications/server/protocol types (e.g., HTTPS connections, SOAP/REST web services, databases, message-oriented middlewares).
- Record, build, and debug mechanisms to speed up the performance engineering process.
- Multi-threading frameworks enable concurrent sampling.



Microbenchmarking

Microbenchmarking specifies **test environments** and **test cases** to compare the performance attributes of **multiple implementations**.

It can also be used to determine whether **a source code change has introduced performance regressions**.

Automated performance testing allows developers to **optimize** source code until reaching the expected requirements.

Java Microbenchmark Harness (JMH) is the de facto standard benchmarking framework available since OpenJDK 1.7. It can be seen as an extension of JUnit, i.e., an **annotation-based framework** that enables control over the execution of test cases.

Microbenchmarking

```
@fork(2)
@Warmup(iterations = 5)
@Measurement(iterations = 10)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)

public class RuntimeSchemaBenchmark {
    private RuntimeSchemaBenchmark<Int1> int1RuntimeSchema = ...;
    private byte[] data_1_int = ...;

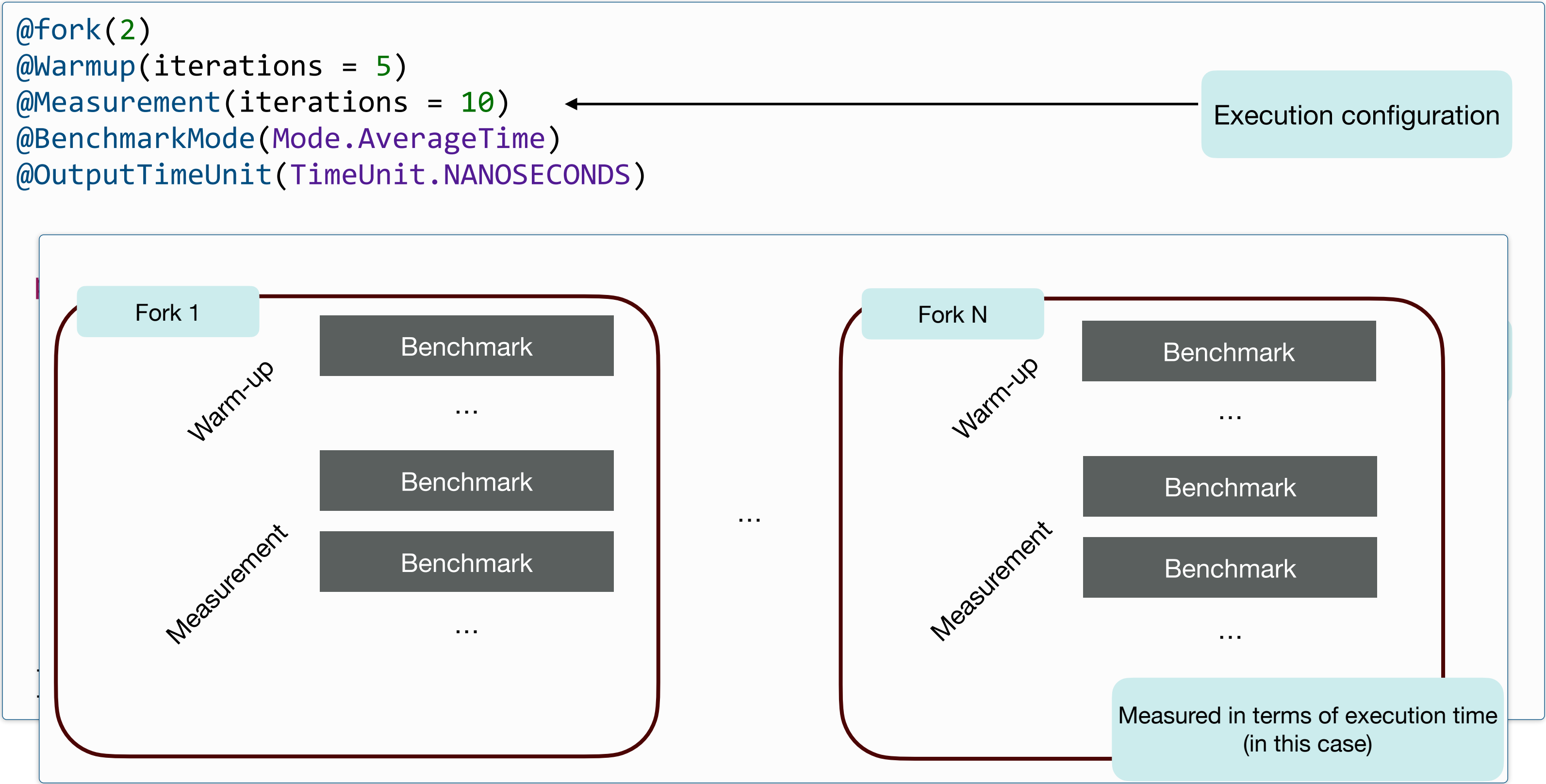
    @Benchmark
    public Int1 runtime_deserialize_1_int_field() throws Exception {
        Int1 int1 = new Int1();
        ProtobufIOUtil.mergeFrom(data_1_int, int1, int1RuntimeSchema);
        return int1;
    }
}
```

Execution configuration

Benchmark

The diagram illustrates the structure of a microbenchmarking class. A light blue box labeled 'Execution configuration' has an arrow pointing to the first five annotations: @fork(2), @Warmup(iterations = 5), @Measurement(iterations = 10), @BenchmarkMode(Mode.AverageTime), and @OutputTimeUnit(TimeUnit.NANOSECONDS). Another light blue box labeled 'Benchmark' has an arrow pointing to the @Benchmark annotation on the runtime_deserialize_1_int_field() method.

Microbenchmarking



Microbenchmarking

To develop and execute microbenchmarks effectively, you should know some common pitfalls of **test environments** and **program executions**.

Environmental Aspects

- Operating system (type, version);
- Virtualization (VMs, containers, cloud);
- Multi-tenancy, i.e., dynamic partitioning of data and configuration so that each client works on a customized virtual instance;
- Background processes;
- Disk capabilities (HDD, SSD, RAID);
- Memory capabilities;
- CPU (clock rate, architecture);
- Caches;
- Environmental variables;
- And many others...

Program Execution

- VM version (JVM);
- Dynamic compilation (JIT, AOT);
- Compiler optimizations;
- Garbage collection options;
- Memory (e.g., stack, heap) sizes;
- Thread scheduler;
- Caches;
- Software pipelining;
- And many others...

Microbenchmarking

To develop and execute microbenchmarks effectively, you should know some common pitfalls of **test environments** and **program executions**.

All these aspects make the measurements **non-deterministic**. Let us consider the case of compiler optimizations.

Static compiler

- Source code;
- Optimizations;
- Executable machine code.

Traditional JIT compiler

- Source code;
- Platform-independent byte code (almost no optimizations);
- Compile on-demand when code is executed.

Modern JIT compiler

- Source code;
- Platform-independent byte code (almost no optimizations);
- Profile code to find “hotspots”, code that is often executed;
- Optimize “hotspots” and execute compiled code.

Examples of dynamic compiler optimizations are method inlining, loop unrolling, branch prediction, dead-code removal, and on-stack replacement.

The optimizations are based on the type of compiler you are using.

Run `java -XX:+PrintCompilation -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining -verbose:gc`

JMH - Basics

```
@Fork(1)
@Threads(1)
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.MICROSECONDS)

public class Basics {

    @Benchmark
    @BenchmarkMode(Mode.Throughput)
    @OutputTimeUnit(TimeUnit.SECONDS)
    public void measureThroughput() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }

    @Benchmark
    @BenchmarkMode(Mode.AverageTime)
    @OutputTimeUnit(TimeUnit.SECONDS)
    public void measureAvgTime() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }
}
```

```
# JMH version: 1.19
# VM version: JDK 1.8.0_102, VM 25.102-b14
# VM invoker: /Library/Java/JavaVirtualMachines/jdk1.8.0_102.jdk/Contents/Home/jre/bin/java
# VM options: <none>
# Warmup: 5 iterations, 1 s each
# Measurement: 5 iterations, 1 s each
# Timeout: 10 min per iteration
# Threads: 1 thread, will synchronize iterations
# Benchmark mode: Throughput, ops/time
# Benchmark: ch.uzh.ifi.seal.ase17.bench.Basics.measureThroughput

# Run progress: 0.00% complete, ETA 00:00:30
# Fork: 1 of 1
# Warmup Iteration   1: 9.701 ops/s
# Warmup Iteration   2: 9.716 ops/s
# Warmup Iteration   3: 9.703 ops/s
# Warmup Iteration   4: 9.659 ops/s
# Warmup Iteration   5: 9.803 ops/s
Iteration    1: 9.716 ops/s
Iteration    2: 9.781 ops/s
Iteration    3: 9.771 ops/s
Iteration    4: 9.682 ops/s
Iteration    5: 9.632 ops/s

Result "ch.uzh.ifi.seal.ase17.bench.Basics.measureThroughput":
 9.716 ±(99.9%) 0.238 ops/s [Average]
 (min, avg, max) = (9.632, 9.716, 9.781), stdev = 0.062
 CI (99.9%): [9.478, 9.954] (assumes normal distribution)
```

Results are always
summarized

JMH - Basics

```
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
```

```
public class DefaultState {
```

```
    double x = Math.PI;
```

```
    @Setup
```

```
    public void prepare() {
        x = Math.PI;
    }
```

```
    @TearDown
```

```
    public void check() {
        assert x > Math.PI : "Nothing changed?"
    }
```

```
    @Benchmark
```

```
    public void measure() {
        x++;
    }
```

```
}
```

Similarly to JUnit, one could define
setup and teardown methods

```
@Warmup(iterations = 5)
@Measurement(iterations = 5)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
```

```
public class Params {
```

```
    @Param({"1", "31", "65", "101", "103", })
    public int arg;
```

```
    @Param({"0", "1", "2", "4", "8", "16", "32"})
    public int certainly;
```

```
    @Benchmark
```

```
    public boolean bench() {
        return BigInteger.valueOf(arg).
            isProbablePrime(certainly);
    }
```

```
}
```

Parameters can be set as well to
provide inputs to the benchmarks.

JMH - Basics

If you play with JMH, you will notice that it has dozens of further features like:

- Safe looping;
- Multithreading, asymmetric execution/groups, synchronized iterations;
- Compiler control (i.e., inline, do not inline, exclude);
- Fine-grained configurations;
- CPU monitoring;
- Slowdowns due to memory issues;
- Benchmark class inheritance (abstract classes);
- Interrupt dead-lock benchmarks;
- Infrastructure parameters for adaptable benchmarks;
- Bulk warmup (first all warmups, then measurements -> type profile skew);
- Regular datasets and branch prediction;
- Additional profiling;
- Different performance due to cache access patterns.

More info: <http://hg.openjdk.java.net/code-tools/jmh/file/tip/jmh-samples/src/main/java/org/openjdk/jmh/samples/>

All results you obtain will be distributions. You will need statistical methods to compare them.

A close-up, angled shot of a white computer keyboard. The central focus is a large, rectangular white key with rounded corners. On this key, there is a dark blue icon of a coffee cup with three wavy lines above it representing steam. Below the icon, the word "Break" is printed in a dark blue, serif font. The key is set against a light-colored, textured keyboard surface. Other keys are visible in the background, including one with a double quote symbol and another with a four-way arrow symbol, but they are out of focus.

Break